RL Factor Timing

Baseline vs Optuna-Tuned PPO Hyperparameters

Kyle Parran • Songnan Lin • Yuhan Guo • Prashanth Bhaskara

2025-12-10

Agenda

1. Motivation & research question
2. Why PPO?
3. Data universe + selection
4. Baselines + Strategies
5. Optuna tuning + comparison
6. Results: held-out + walk-forward
7. Takeaways + next steps

What you'll see

A realistic trading simulator.
Honest OOS testing via walk-forward windows.
Baseline vs tuned performance summary.

Research question & goals

What we're testing

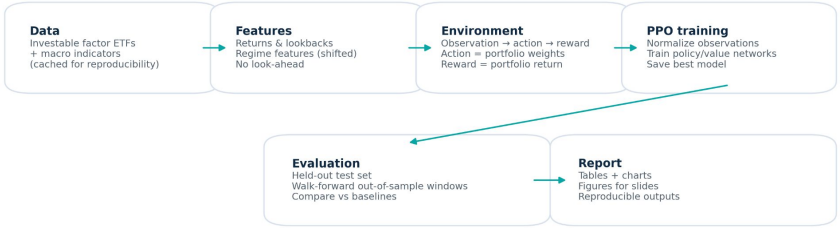
Research question

Can PPO learn dynamic factor allocations that outperform robust baselines?

Implementation

- Factor timing requires optimal allocation across multiple factors
- Traditional factor timing relies on simple heuristics or rules
- Reinforcement learning can learn complex factor timing strategies
- RL performance heavily depends on hyperparameter selection
- Automated tuning can potentially improve RL agent performance

End-to-end research pipeline



PPO agent methodology

Algorithm: Proximal Policy Optimization (PPO) for Factor Timing

Why PPO:

Exploration vs. exploitation: PPO encourages the agent to explore new allocations while still favoring the weight choices that historically performed well.

Continuous weights: Our portfolio weights are continuous values, so we need an algorithm that works naturally with continuous actions. PPO fits this perfectly.

Stable learning: PPO is designed to avoid “wild” updates, which is important because financial returns are noisy and unstable during training

How PPO Learns in This Project

Observation:The agent sees a vector of features (factor returns, macro variables, volatility, etc.).

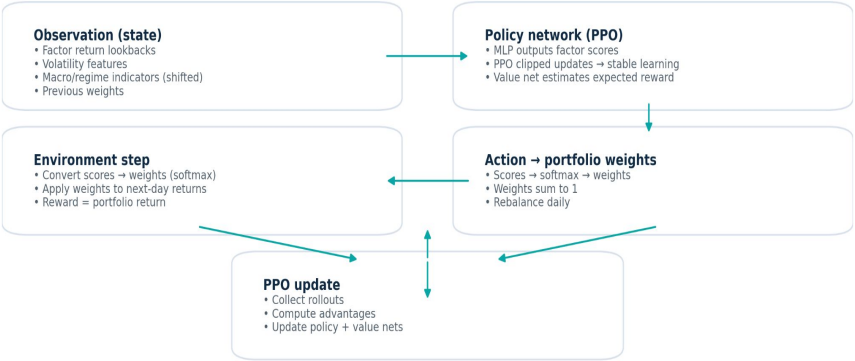
Action (Portfolio Weights): The PPO policy network outputs a set of factor weights between 0 and 1, which define the day’s allocation.

Environment Step: The portfolio experiences the returns of that allocation producing a profit or loss.

Reward:Daily portfolio return becomes the reward signal. PPO tries to maximize long-term cumulative reward.

Policy Update: PPO compares actions that worked well vs ones that didn’t and nudges policies towards the good ones.

PPO agent methodology — how training works in this project



Datasets

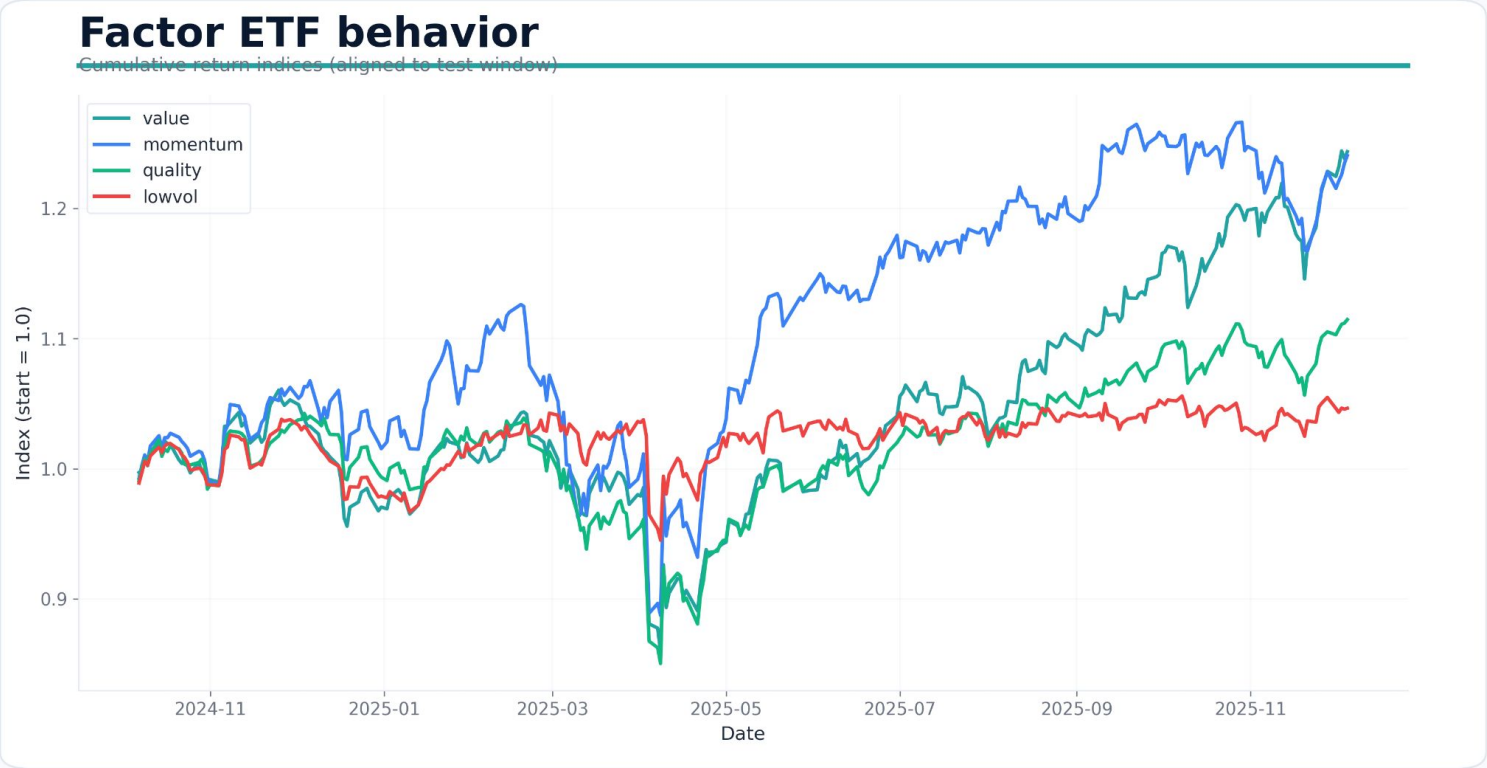
Investable factor proxies + macro regime inputs

Primary Dataset:
- Contains daily financial time-series data from January 3, 2017, to October 30, 2025
- Covers multiple asset classes:
  - Traditional Indices & Rates
  - Exchange Traded Funds
  - Cryptocurrencies
  - Specialized Indices
  - Fama-French 3 Factors

Factor ETFs (Investment Targets):
- **Value**: VLUE - **Momentum**: MTUM - **Quality**: QUAL - **Low-Volatility**: USMV
- **Source**: Fetched from Stooq (external data provider)
- Daily price data converted to returns for modeling

Strategy	Ticker
Value	VLUE
Momentum	MTUM
Quality	QUAL
Low Vol	USMV

Datasets


Factor ETF behavior
Cumulative return indices (aligned to test window)
value
momentum
quality
lowvol
Index (start = 1.0)
1.2
1.1
1.0
0.9
2024-11
2025-01
2025-03
2025-05
2025-07
2025-09
2025-11
Date

Datasets

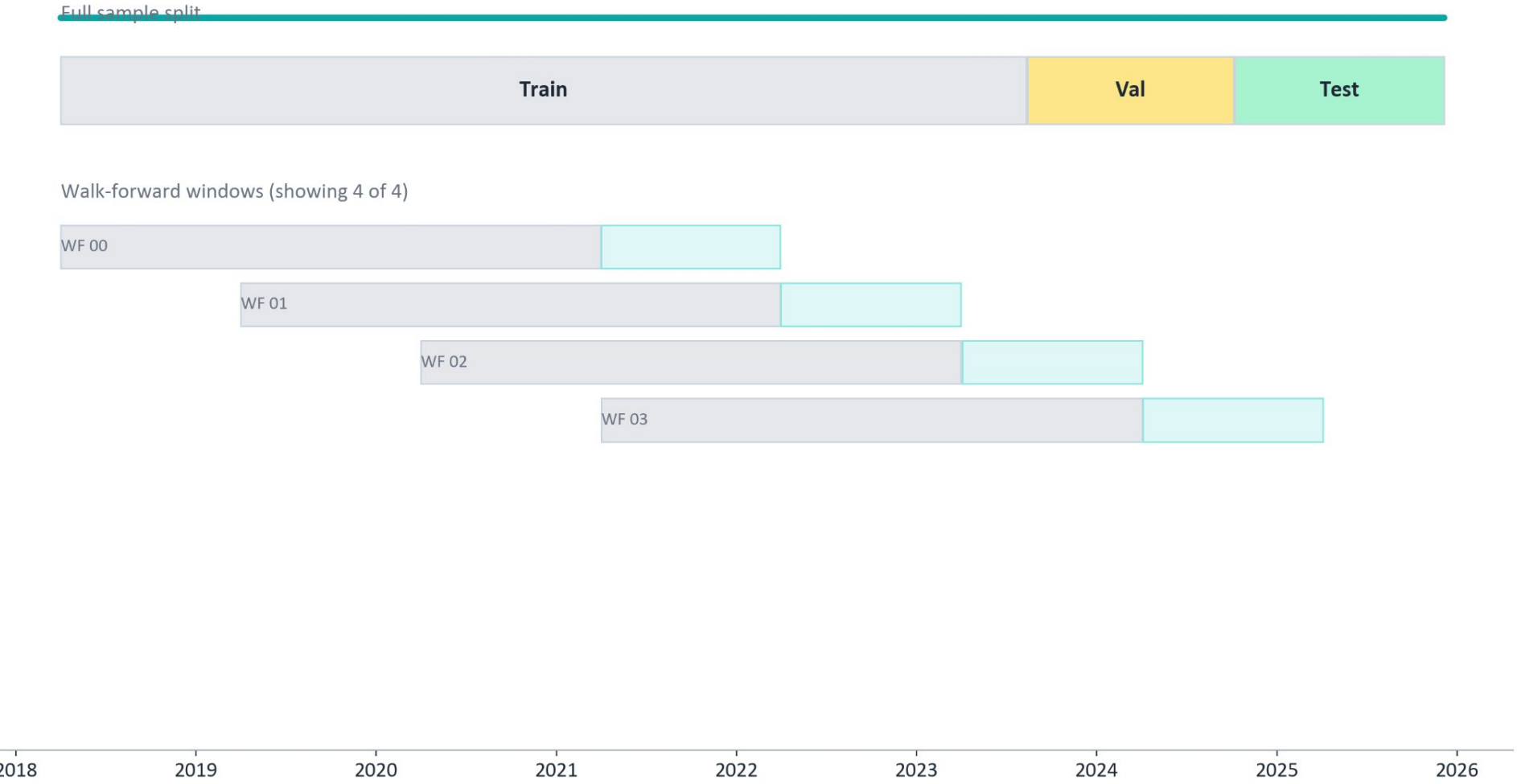
Time Period & Splits

- Full Dataset: January 3, 2017 to October 30, 2025 (daily frequency)
- Train Set: 70% (2017-2023)
- Validation Set: 15% (2023-2024)
- Test Set: 15% (2024-2025)
- Walk-forward: Rolling 756-day training windows with 252-day test periods (4 windows)

Data Preprocessing

- Forward-fill missing macro data to handle gaps
- Convert prices to returns
- Features computed with proper lookback
- Create engineered features: momentum (5, 20, 60-day), volatility (20, 60-day), cross-sectional z-scores

Datasets — Train/Val/Test Split + Walk-Forward OOS Windows



Trading Strategies

Strategy 1: Equal Weight

Puts 25% of money into each of the 4 factors, every single day. Never changes

Strategy 2: Inverse Volatility

Allocates MORE money to factors with LOWER volatility (less swingy), LESS money to high-volatility factors

Strategy 3: Score-Based

Creates a "score" for each factor = (Momentum - Volatility), then allocates more to factors with higher scores.

Strategy 4: Reinforcement Learning (RL) Agent

An AI agent that LEARNS from experience which factors to invest in based on market conditions. It observes many features and decides allocations.

Strategy 5: Tuned Reinforcement Learning (RL) Agent

Uses Optuna to optimize the RL (PPO) model and creates models with different hyperparameters

Trading Strategies — What Each Method Uses

	Dynamic weights	Uses returns	Uses volatility	Uses macro/regime	Learns policy	Hyperparam tuned
Equal Weight						
Inverse Vol	?		?			
Score-Based	?	?	?			
RL (PPO)	?	?	?	?	?	
RL + Optuna	?	?	?	?	?	?

Interpretation: moving down the list increases adaptivity and modeling complexity.
RL learns allocations from features; Optuna tunes the learning process (not the market).

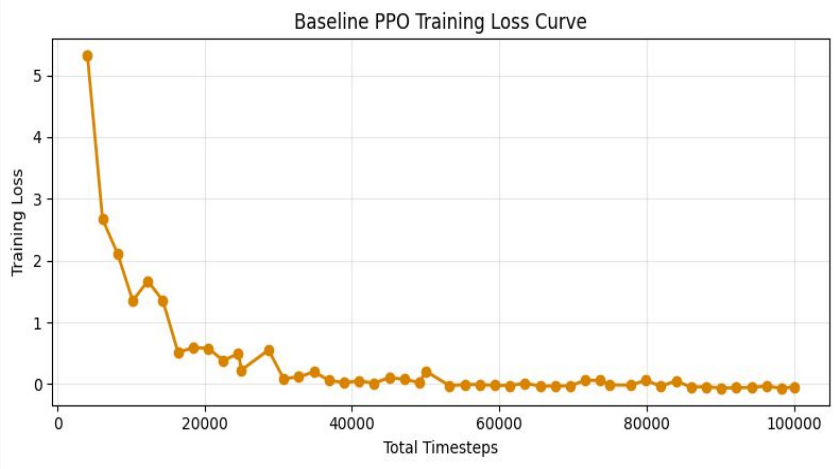
Training Output Interpretation

Baseline Training Setup:

- Timesteps: **100,000**
- Eval Frequency: **25,000 steps**
- Seed: **7**
- Default PPO hyperparameters (as provided by Stable-Baselines3)
- Environment includes:
  - Factor exposure signals
  - Risk penalty
  - Transaction costs
- Purpose: establish a **minimum acceptable performance floor.**

Issue:

baseline **training is stable**, but the strategy is not strong enough for high Sharpe timing.



Metric	Value	Interpretation
approx_kl	0.0106	Stable policy update; PPO behaving correctly
explained_variance	0.99	Value function fits signal well
entropy_loss	-5.45	Decreasing entropy → less randomness
value_loss	0.265	Normal critic training behavior

Optuna Tuning Method

What Optuna Changes

Optuna automatically searches for better-performing PPO hyperparameters, including:

- Learning rate
- Network architecture (policy/value net sizes)
- Entropy coefficient
- Discount factor γ
- λ (GAE smoothing)
- Batch size / n_steps

Objective

- Maximize **Sharpe ratio** on validation set
- Avoid overfitting by evaluating across rolling windows

How It Improves Training

- Finds settings that reduce variance
- Produces smoother policy updates
- Enhances stability in volatile regimes
- Learns better regime-switching behavior

Optuna Tuning Loop (Conceptual)

Goal: find hyperparameters that improve out-of-sample behavior (e.g., Sharpe on validation) while keeping training stable. This tunes the *learning process*, not the market itself.

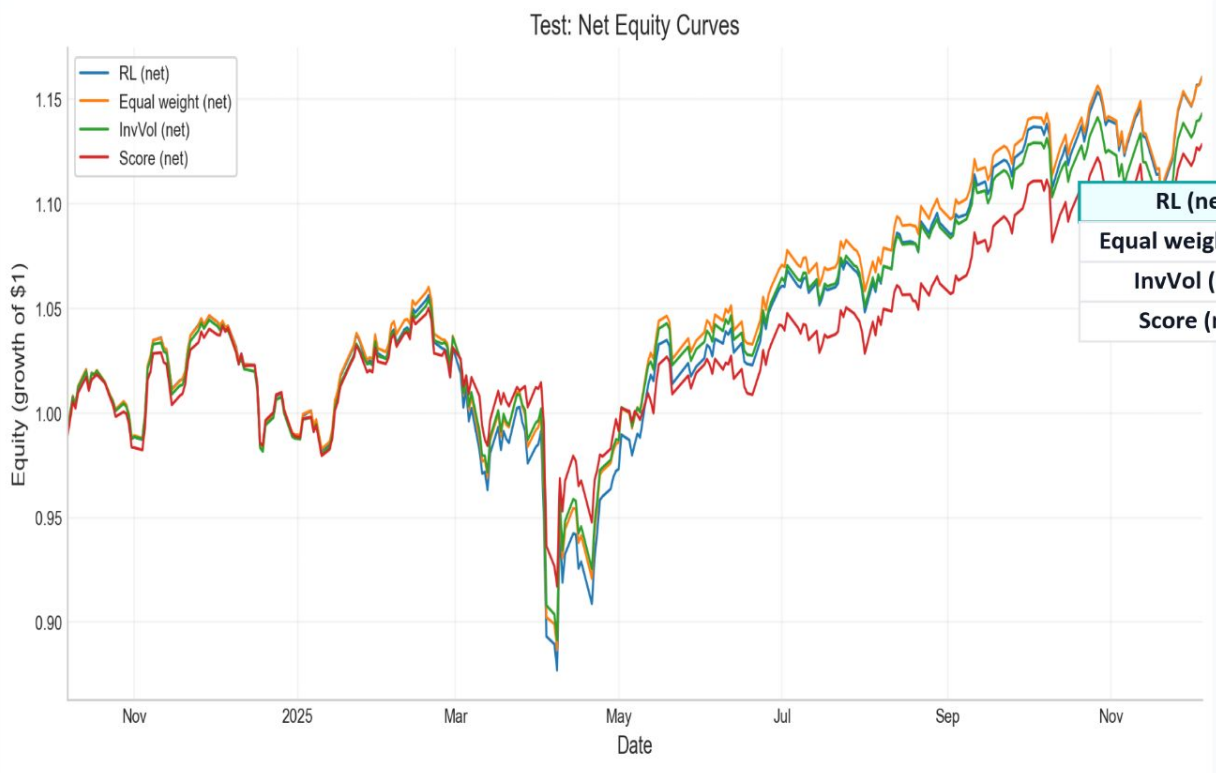
Optimized (tuned) PPO Configurations

Tuned Training Setup

- Hyperparameter knobs tuned with Optuna:
  - learning rate
  - entropy coefficient
  - budget penalty (risk penalty in the reward)
  - turnover penalty (transaction cost penalty in the reward)
- Tuning phase – many short runs:
  - Optuna runs 15 trials
  - each trial trains a PPO agent for 40,000 timestep
- Final training with best hyperparameters:
  - re-train PPO from scratch
  - **120,000 timesteps** on the training data

Results — Untuned RL Agent (Held-out Test)

Net-of-cost equity (left) + performance summary (right)



Performance snapshot

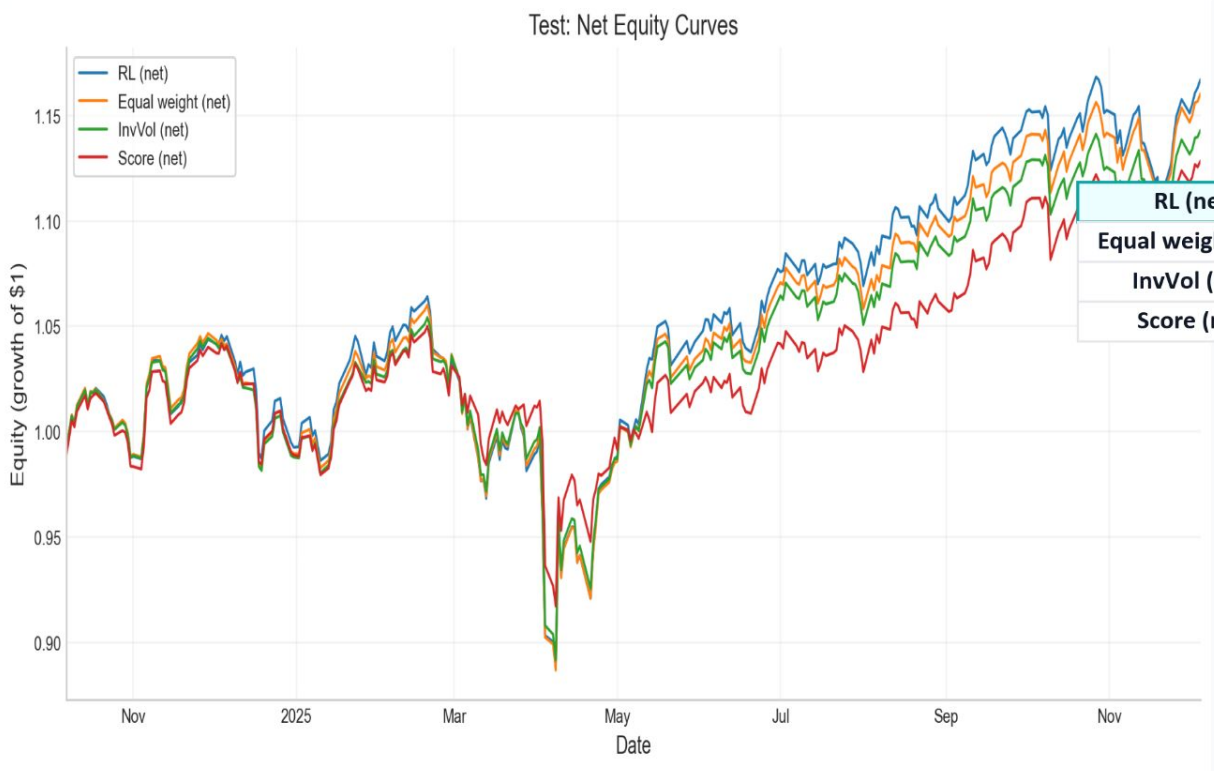
	Total	CAGR	Max DD	Sharpe
RL (net)	16.1%	13.8%	-17.0%	0.88
Equal weight (net)	16.0%	13.8%	-16.4%	0.89
InvVol (net)	14.3%	12.3%	-15.4%	0.83
Score (net)	12.8%	11.1%	-12.7%	0.83

Key takeaways

- Sharpe rank: #2 (best: Equal weight (net))
- Total return: 16.1%
- Max drawdown: -17.0%

Results — Optuna Tuned RL Agent (Held-out Test)

Net-of-cost equity (left) + performance summary (right)



Performance snapshot

	Total	CAGR	Max DD	Sharpe
RL (net)	16.7%	14.4%	-16.6%	0.90
Equal weight (net)	16.0%	13.8%	-16.4%	0.89
InvVol (net)	14.3%	12.3%	-15.4%	0.83
Score (net)	12.8%	11.1%	-12.7%	0.83

Key takeaways

- Sharpe rank: #1 (best: RL (net))
- Total return: 16.7%
- Max drawdown: -16.6%

Findings

Baseline

Optuna Tuned

Held out Test

Strategy	# days	Total Return	Sharpe
RL (net)	290	16.1%	0.88
Equal weight (net)	290	16.0%	0.89
InvVol (net)	290	14.3%	0.83
Score (net)	290	12.8%	0.83

Sharpe, Return Increases,

Strategy	# days	Total Return	Sharpe
RL (net)	290	16.7%	0.90
Equal weight (net)	290	16.0%	0.89
InvVol (net)	290	14.3%	0.83
Score (net)	290	12.8%	0.83

Sharpe, Return Decreases,

Sharpe, Return, Decreases

Full Time Period

Strategy	# days	Total Return	Sharpe
RL (net)	1008	14.1%	0.29
Equal weight (net)	1008	19.3%	0.36
InvVol (net)	1008	21.1%	0.39
Score (net)	1008	23.0%	0.45

Strategy	# days	Total Return	Sharpe
RL (net)	1008	14.1%	0.29
Equal weight (net)	1008	19.3%	0.36
InvVol (net)	1008	21.1%	0.39
Score (net)	1008	23.0%	0.45

Findings Summarized

- In the Held out Test, the RL strategy's performance improves as we transition from Baseline to Optuna tuned agents.

- The RL net strategy ultimately has very similar performance to the benchmark strategies

- The Optuna Tuned RL strategy has the best performance amongst all test strategies.

Conclusion

Summary of Findings:

- Automated hyperparameter tuning improved RL agent performance
- Tuned model achieved better returns and risk-adjusted metrics compared to baseline
- RL approach is competitive with traditional factor timing methods

Key Takeaways:

- Hyperparameter tuning significantly impacts RL performance in finance applications
- Automated optimization methods (Optuna) can effectively identify superior hyperparameter configurations
- RL-based factor timing is a viable approach for portfolio management

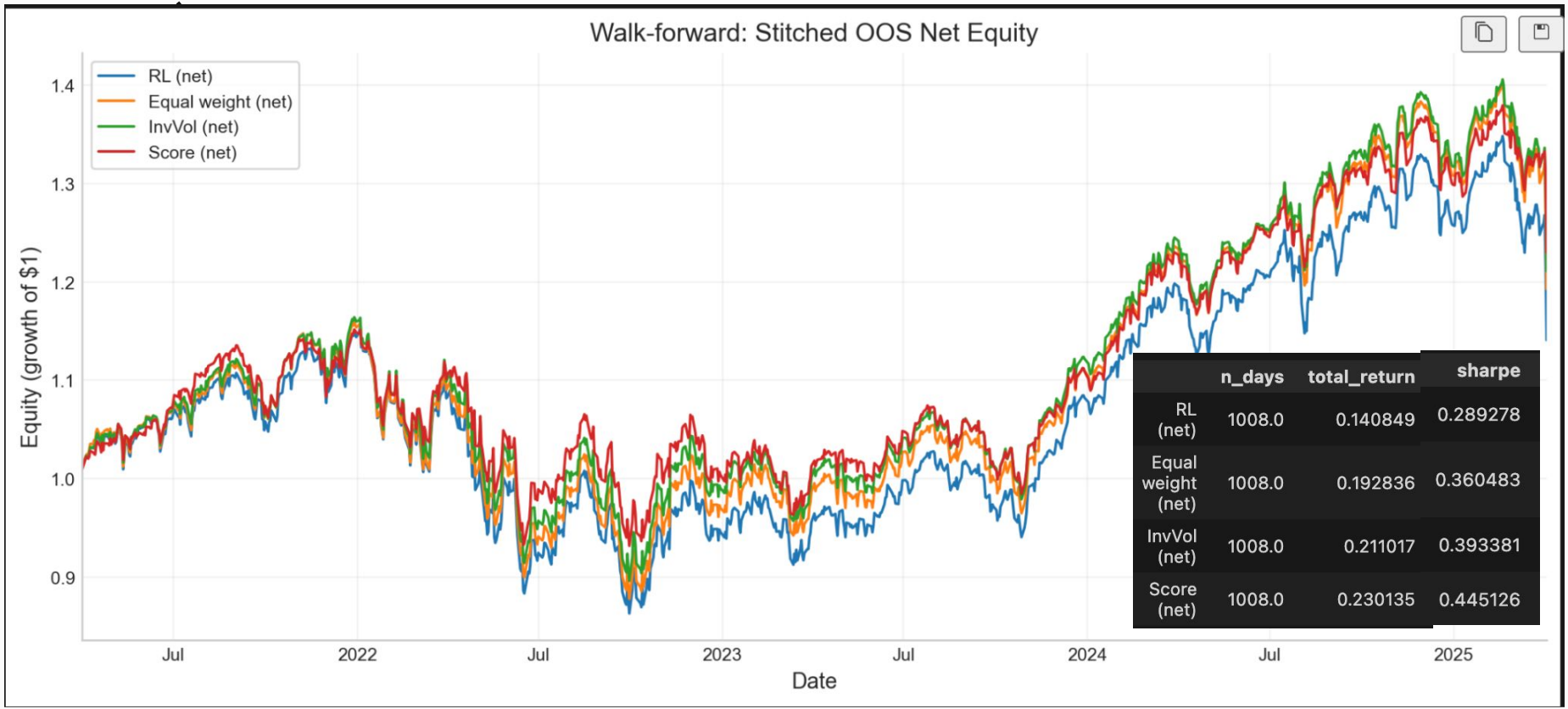
Limitations & Future Work:

- Computational resources required for hyperparameter search
- Extended hyperparameter search space, additional factors, ensemble methods.

Thank You!

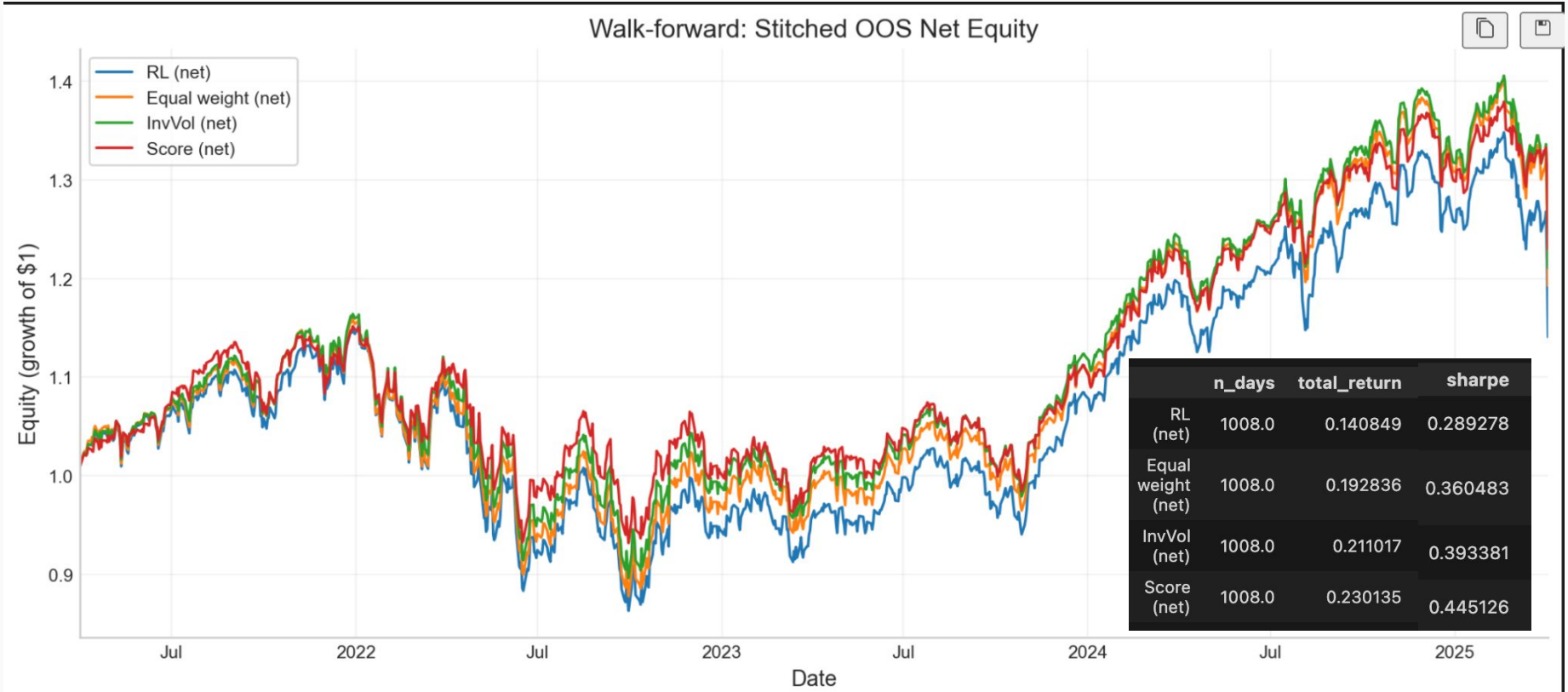
Q&A

Appendix - Untuned RL Agent (Full Time)



	n_days	total_return	sharpe
RL (net)	1008.0	0.140849	0.289278
Equal weight (net)	1008.0	0.192836	0.360483
InvVol (net)	1008.0	0.211017	0.393381
Score (net)	1008.0	0.230135	0.445126

Appendix - Optuna Tuned RL Agent (Out of Sample)



	n_days	total_return	sharpe
RL (net)	1008.0	0.140849	0.289278
Equal weight (net)	1008.0	0.192836	0.360483
InvVol (net)	1008.0	0.211017	0.393381
Score (net)	1008.0	0.230135	0.445126